TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI

Đề cương bài giảng

Lập trình hướng đối tượng

Ngành : Công nghệ thông tin
Trình độ đào tạo: Đại học
Người biên soạn : Khoa CNTT

(Lưu hành nội bộ)

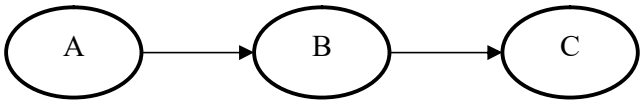


Hà Nội – 5 / 2021

Bài 5. Phương thức ảo, tính đa hình, khuôn mẫu và biệt lệ.

1. Con trỏ đối tượng và các phương thức tĩnh

Giả sử có 3 lớp A, B, C kế thừa nhau theo cây thứ bậc sau:



Tức lớp B kế thừa lớp A, Lớp C lại kế thừa trực tiếp lớp B.

Nếu ta khai báo một con trỏ đối tượng P thuộc lớp A (A * P) thì: P có thể chứa địa chỉ của các đối tượng thuộc lớp A hoặc B hoặc C.

VD: Ta khai báo: `A a, *P; B b; C c;` thì ta có thể viết: `P = &a;` hoặc `P = & b`; hoặc `P = & c;`

Xét trường hợp cả 3 lớp A, B, C đều có cùng một phương thức: cùng tên, cùng danh sách các đối, chỉ khác nhau về nội dung phương thức. Khi đó, nếu ta viết: P → <Tên phương thức>; thì phương thức nào trong 3 phương thức của 3 lớp sẽ được gọi?

Câu trả lời là: Chỉ có phương thức của lớp A sẽ được gọi, cho dù P có trỏ tới đối tượng thuộc lớp B và C.

VD: Xét đoạn trình mô tả 3 lớp A, B, C kế thừa nhau theo cây thứ bậc trên. Cả 3 lớp đều có phương thức giống nhau là phương thức nhap().

```
class A
{
      int a;
public:
      void nhap()
      {
            cout<<"Nhap a "; cin>>a;
      }
};
class B: public A
{
      int b;
public:
      void nhap()
      {
            cout<<" Nhap b "; cin>>b;
      }
};
class C: public B
{
      int c;
```

```
public:
      void nhap()
      {
            cout<<" Nhap c "; cin>>c;
      }
};
```

Tại hàm main(), ta khai báo 3 đối tượng thuộc 3 lớp A, B, C và một con trỏ p thuộc lớp A. Khi đó, mặc dù p trỏ tới đối tượng của lớp B, C những khi viết p → nhap() thì phương thức nhap() của lớp A vẫn được gọi.

```
void main()
{
      A d1, *p;
      B d2;
      C d3;
      p = & d2;
      p->nhap();//van la phuong thuc nhap() cua lop A
      p = & d3;
      p->nhap();//van la phuong thuc nhap() cua lop A
}
```

2. Phương thức ảo và ý nghĩa của phương thức ảo

Trong nhiều trường hợp, ta mong muốn:

Khi con trỏ đối tượng p thuộc lớp A đang chứa địa chỉ của một đối tượng thuộc lớp B hoặc C mà ta viết: P → nhap(); thì sẽ truy cập tới phương thức nhap() của lớp B hoặc C.

Muốn được như vậy thì phương thức nhap() của 3 lớp A, B, C phải là phương thức ảo.

Các phương thức viết theo kiểu thông thường đều là các phương thức tĩnh. Phương thức nhap() trong ví dụ trên là phương thức tĩnh.

Cách chuyển phương thức tĩnh thành phương thức ảo:

Cách 1: Thêm từ khoá virtual vào trước phương thức tĩnh của cả tất cả các lớp (cơ sở và dẫn xuất).

VD: Đoạn trình sau chuyển phương thức tĩnh nhap() thành phương thức ảo:

```
class A
{
      int a;
public:
      virtual void nhap()
      {
            cout<<"Nhap a "; cin>>a;
      }
};

class B: public A
```

```
{
      int b;
public:
      virtual void nhap()
      {
           cout<<" Nhap b "; cin>>b;
      }
};

class C: public B
{
      int c;
public:

      virtual void nhap()
      {
           cout<<" Nhap c "; cin>>c;
      }
};
```

Cách 2: Chỉ cần thêm từ khoá virtual vào trước phương thức tĩnh của lớp cơ sở.

```
class A
{
      int a;
public:
      virtual void nhap()
      {
           cout<<"Nhap a "; cin>>a;
      }
};
class B: public A
{
      int b;
public:
      void nhap()
      {
           cout<<" Nhap b "; cin>>b;
      }
};
class C: public B
{
      int c;
public:
      void nhap()
      {
           cout<<" Nhap c "; cin>>c;
      }
};
```

Khi đó, trong chương trình chính, nếu p trỏ tới đối tượng của lớp nào thì phương thức nhap() của đối tượng thuộc lớp đó sẽ được gọi khi ta viết p → nhap();

```
void main()
{
     A d1, *p;
     B d2;
     C d3;
     p = & d2;
     p->nhap();//phuong thuc nhap() cua lop B duoc goi
     p = & d3;
     p->nhap();//phuong thuc nhap() cua lop C duoc goi
}
```

3. Phương thức ảo và sự kết nối động

Trong ví dụ trên, cùng một lời gọi phương thức : p → nhap(); nhưng có thể truy cập tới phương thức ảo của lớp A hoặc B hoặc C. Lời gọi p → nhap(); tương ứng với 3 phương thức nhap() khác nhau.

Khi ta sử dụng phương thức ảo, rõ ràng là: cùng một con trỏ thuộc lớp cơ sở, cùng một lời gọi phương thức nhưng lời gọi đó lại tương ứng với nhiều phương thức khác nhau. Ta gọi đó là sự tương ứng bội hay tính đa hình.

Lời gọi p → nhap(); có thể kết nối tới phương thức nhap() của lớp A hoặc lớp B, hoặc lớp C. Điều đó có nghĩa là lời gọi đó không liên kết cứng tới một phương thức nhap() nào mà sự liên kết đó là động.

Như vậy, khi sử dụng phương thức ảo thì ta có thể liên kết động từ một lời gọi phương thức tới nhiều phương thức cùng tên, cùng bộ đối số. Tính chất như vậy của phương thức ảo gọi là sự kết nối động.

4. Ví dụ về sử dụng phương thức ảo

Xây dựng lớp Cây gồm các thuộc tính: Chiều cao, độ tuổi, chu vi tán và các phương thức:

- Phương thức nhập: nhập các giá trị cho các thuộc tính của lớp Cây.
- Phương thức xuất: xuất các giá trị của các thuộc tính thuộc lớp Cây lên màn hình.

Xây dựng lớp Cây cảnh, ngoài các thuộc tính của lớp Cây còn có các thuộc tính: Giá thành, chủng loại và các phương thức:

- Phương thức nhập: nhập các giá trị cho các thuộc tính của lớp Cây cảnh.
- Phương thức xuất: xuất các giá trị của các thuộc tính thuộc lớp Cây cảnh lên màn hình.

Viết chương trình chính khai báo 2 đối tượng thuộc 2 lớp trên và một con trỏ thuộc lớp Cây. Dùng con trỏ này để nhập, xuất các thuộc tính của hai đối tượng trên.

```
class Cay
{
public:
 float Chieucao;
 float Dotuoi;
 float CVTan;
public:
 virtual void nhap()
 {
      cout<<"Nhap thong tin cho Cay "<<endl;
      cout<<"Chieu cao "; cin>>Chieucao;
      cout<<"Do tuoi "; cin>>Dotuoi;
      cout<<"Chu vi tan "; cin>>CVTan;
 }
 virtual void xuat()
 {
      cout<<"Thong tin cua lop Cay "<<endl;
      cout<<"Chieu cao: "<<Chieucao<<endl;
      cout<<"Do tuoi: "<<Dotuoi<<endl;
      cout<<"Chu vi tan: "<<CVTan<<endl;
 }
};
class Caycanh: public Cay
{
 float Giathanh;
 char Chungloai[30];
public:
 void nhap()
 {
      cout<<"Nhap thong tin cho Cay canh"<<endl;
      cout<<"Chieu cao "; cin>>Chieucao;
      cout<<"Do tuoi "; cin>>Dotuoi;
      cout<<"Chu vi tan "; cin>>CVTan;
      cout<<"Gia thanh "; cin>>Giathanh;
      cout<<"Chung loai "; gets(Chungloai);
 }
 void xuat()
 {
      cout<<"Thong tin cua lop Cay Canh "<<endl;
      cout<<"Chieu cao: "<<Chieucao<<endl;
      cout<<"Do tuoi: "<<Dotuoi<<endl;
      cout<<"Chu vi tan: "<<CVTan<<endl;
      cout<<"Gia thanh: "<<Giathanh<<endl;
      cout<<"Chung loai: "<<Chungloai<<endl;
 }
};
void main()
{
      Cay a, *p;
      Caycanh b;
      p = &a;
```

```
        p->nhap();//Nhap cua lop Cay
        p=&b;
        p->nhap();//Nhap cua lop Caycanh
        p=&a;
        p->xuat();//Xuat cua lop Cay
        p=&b;
        p->xuat();//Xuat cua lop Caycanh
        getch();
}
```

5. Lớp cơ sở trừu tượng và các thành phần ảo

Trong nhiều trường hợp, ta chỉ muốn dùng con trỏ của lớp cơ sở để truy cập tới các phương thức ảo của các lớp dẫn xuất. Ngoài ra, rất ít khi dùng con trỏ lớp cơ sở để truy cập tới phương thức ảo của chính lớp này.

Tuy nhiên, để con trỏ của lớp cơ sở có thể truy cập các phương thức ảo của các lớp dẫn xuất thì lớp cơ sở cũng phải có phương thức ảo này. Từ đây xuất hiện một khả năng: Phương thức ảo của lớp cơ sở có thể chỉ được định nghĩa hình thức mà không được dùng. Khi đó thân của phương thức ảo này không cần có bất cứ dòng lệnh nào.

Một phương thức ảo của lớp cơ sở mà trong thân của nó không thực thi một lệnh nào (trừ return) gọi là phương thức thuần ảo.

Lớp cơ sở có phương thức thuần ảo gọi là lớp cơ sở trừu tượng.

Khi thiết kế phần mềm hướng đối tượng, ta luôn xác định được các lớp có thể có trong phần mềm và cây thứ bậc thể hiện sự kế thừa của các lớp. Người ta thường tạo ra các lớp cơ sở trừu tượng, trong đó có các phương thức thuần ảo. Những phương thức như vậy không thực hiện một công việc nào mà chỉ dùng để tạo phương thức ảo cho các phương thức cùng tên trong các lớp dẫn xuất. Từ đó, chỉ cần khai báo một con trỏ thuộc lớp cơ sở trừu tượng này, ta có thể dùng con trỏ đó để truy cập tới các phương thức ảo của các lớp dẫn xuất. Từ đây, tạo ra sự linh hoạt trong truy cập các lớp dẫn xuất.

Phương thức ảo chỉ được tạo ra sau khi đã hình thành đối tượng, do vậy, phương thức khởi tạo không thể là phương thức ảo nhưng phương thức huỷ bỏ có thể là phương thức ảo. Ngoài ra, phương thức toán tử cũng có thể là phương thức ảo.

ưu nhược điểm của phương thức ảo:

- Chương trình sử dụng nhiều phương thức ảo sẽ linh hoạt hơn trong sự truy cập các phương thức cùng tên của các lớp dẫn xuất.
- Việc thực thi chương trình sẽ chậm hơn.

- Tốn nhiều bộ nhớ hơn do phải tạo ra một bảng chỉ mục của các phương thức ảo.

Khi nào dùng phương thức ảo?

Ta chỉ thiết kế các lớp có phương thức ảo khi:

- Có sự kế thừa giữa các lớp.
- Các lớp trong cây thứ bậc có các phương thức cùng tên, cùng đối số, lớp cơ sở ban đầu (lớp gốc) bắt buộc cũng phải có phương thức này.
- Bản chất của cây thứ bậc đòi hỏi cần có phương thức ảo.